Frequently Asked Questions (FAQ)

DataFrame memory usage

The memory usage of a `DataFrame` (including the index) is shown when calling the `info()`. A configuration option, `display.memory_usage` (see the list of options), specifies if the `DataFrame` memory usage will be displayed when invoking the `info()` method.

For example, the memory usage of the `DataFrame` below is shown when calling `info()`:

```
In [1]: dtypes = [
   ...:     "int64",
   ...:     "float64",
   ...:     "datetime64[ns]",
   ...:     "timedelta64[ns]",
   ...:     "complex128",
   ...:     "object",
   ...:     "bool",
   ...: ]
   ...:

In [2]: n = 5000

In [3]: data = {t: np.random.randint(100, size=n).astype(t) for t in dtypes}

In [4]: df = pd.DataFrame(data)

In [5]: df["categorical"] = df["object"].astype("category")

In [6]: df.info()
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   int64            5000 non-null   int64
 1   float64          5000 non-null   float64
 2   datetime64[ns]   5000 non-null   datetime64[ns]
 3   timedelta64[ns]  5000 non-null   timedelta64[ns]
 4   complex128       5000 non-null   complex128
 5   object           5000 non-null   object
 6   bool             5000 non-null   bool
 7   categorical      5000 non-null   category
dtypes: bool(1), category(1), complex128(1), datetime64[ns](1), float64(1), int64(1), o
memory usage: 284.1+ KB
```

The `+` symbol indicates that the true memory usage could be higher, because pandas does not count the memory used by values in columns with `dtype=object`.

Passing `memory_usage='deep'` will enable a more accurate memory usage report, accounting for the full usage of the contained objects. This is optional as it can be expensive to do this deeper introspection.

```
In [7]: df.info(memory_usage="deep")
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   int64            5000 non-null   int64
 1   float64          5000 non-null   float64
 2   datetime64[ns]   5000 non-null   datetime64[ns]
 3   timedelta64[ns]  5000 non-null   timedelta64[ns]
 4   complex128       5000 non-null   complex128
 5   object           5000 non-null   object
 6   bool             5000 non-null   bool
 7   categorical      5000 non-null   category
dtypes: bool(1), category(1), complex128(1), datetime64[ns](1), float64(1), int64(1), o
memory usage: 420.8 KB
```

By default the display option is set to `True` but can be explicitly overridden by passing the `memory_usage` argument when invoking `info()`.

The memory usage of each column can be found by calling the `memory_usage()` method. This returns a `Series` with an index represented by column names and memory usage of each column shown in bytes. For the `DataFrame` above, the memory usage of each column and the total memory usage can be found with the `memory_usage()` method:

```
In [8]: df.memory_usage()
Out[8]:
Index                 132
int64               40000
float64             40000
datetime64[ns]      40000
timedelta64[ns]     40000
complex128          80000
object              40000
bool                 5000
categorical          5800
dtype: int64

# total memory usage of dataframe
In [9]: df.memory_usage().sum()
Out[9]: np.int64(290932)
```

By default the memory usage of the `DataFrame` index is shown in the returned `Series`, the memory usage of the index can be suppressed by passing the `index=False` argument:

```
In [10]: df.memory_usage(index=False)
Out[10]:
int64               40000
float64             40000
datetime64[ns]      40000
timedelta64[ns]     40000
complex128          80000
object              40000
bool                 5000
categorical          5800
dtype: int64
```

The memory usage displayed by the `info()` method utilizes the `memory_usage()` method to determine the memory usage of a `DataFrame` while also formatting the output in human-readable units (base-2 representation; i.e. 1KB = 1024 bytes).

See also Categorical Memory Usage.

Using if/truth statements with pandas

pandas follows the NumPy convention of raising an error when you try to convert something to a `bool`. This happens in an `if`-statement or when using the boolean operations: `and`, `or`, and `not`. It is not clear what the result of the following code should be:

```
>>> if pd.Series([False, True, False]):
...     pass
```

Should it be `True` because it's not zero-length, or `False` because there are `False` values? It is unclear, so instead, pandas raises a `ValueError`:

```
In [11]: if pd.Series([False, True, False]):
   ....:     print("I was true")
   ....:
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-11-5c782b38cd2f> in ?()
----> 1 if pd.Series([False, True, False]):
      2     print("I was true")

~/work/pandas/pandas/pandas/core/generic.py in ?(self)
   1511     @final
   1512     def __bool__(self) -> NoReturn:
-> 1513         raise ValueError(
   1514             f"The truth value of a {type(self).__name__} is ambiguous. "
   1515             "Use a.empty, a.bool(), a.item(), a.any() or a.all()."
   1516         )

ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(),
```

You need to explicitly choose what you want to do with the `DataFrame`, e.g. use `any()`, `all()` or `empty()`. Alternatively, you might want to compare if the pandas object is `None`:

```
In [12]: if pd.Series([False, True, False]) is not None:
   ....:     print("I was not None")
   ....:
I was not None
```

Below is how to check if any of the values are `True`:

```
In [13]: if pd.Series([False, True, False]).any():
   ....:     print("I am any")
   ....:
I am any
```

Bitwise Boolean

Bitwise boolean operators like `==` and `!=` return a boolean `Series` which performs an element-wise comparison when compared to a scalar.

```
In [14]: s = pd.Series(range(5))

In [15]: s == 4
Out[15]:
0    False
1    False
2    False
3    False
4     True
dtype: bool
```

See boolean comparisons for more examples.

Using the `in` operator

Using the Python `in` operator on a `Series` tests for membership in the **index**, not membership among the values.

```
In [16]: s = pd.Series(range(5), index=list("abcde"))

In [17]: 2 in s
Out[17]: False

In [18]: 'b' in s
Out[18]: True
```

If this behavior is surprising, keep in mind that using `in` on a Python dictionary tests keys, not values, and `Series` are dict-like. To test for membership in the values, use the method `isin()`:

```
In [19]: s.isin([2])
Out[19]:
a    False
b    False
c     True
d    False
e    False
dtype: bool

In [20]: s.isin([2]).any()
Out[20]: np.True_
```

For `DataFrame`, likewise, `in` applies to the column axis, testing for membership in the list of column names.

Mutating with User Defined Function (UDF) methods

This section applies to pandas methods that take a UDF. In particular, the methods `DataFrame.apply()`, `DataFrame.aggregate()`, `DataFrame.transform()`, and `DataFrame.filter()`.

It is a general rule in programming that one should not mutate a container while it is being iterated over. Mutation will invalidate the iterator, causing unexpected behavior. Consider the example:

```
In [21]: values = [0, 1, 2, 3, 4, 5]

In [22]: n_removed = 0

In [23]: for k, value in enumerate(values):
   ....:     idx = k - n_removed
   ....:     if value % 2 == 1:
   ....:         del values[idx]
   ....:         n_removed += 1
   ....:     else:
   ....:         values[idx] = value + 1
   ....:

In [24]: values
Out[24]: [1, 4, 5]
```

One probably would have expected that the result would be `[1, 3, 5]`. When using a pandas method that takes a UDF, internally pandas is often iterating over the `DataFrame` or other pandas object. Therefore, if the UDF mutates (changes) the `DataFrame`, unexpected behavior can arise.

Here is a similar example with `DataFrame.apply()`:

```
In [25]: def f(s):
   ....:     s.pop("a")
   ....:     return s
   ....:

In [26]: df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})

In [27]: df.apply(f, axis="columns")
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~/work/pandas/pandas/pandas/core/indexes/base.py:3641, in Index.get_loc(self, key)
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File ~/work/pandas/pandas/pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine

File ~/work/pandas/pandas/pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHa

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHa

KeyError: 'a'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[27], line 1
----> 1 df.apply(f, axis="columns")

File ~/work/pandas/pandas/pandas/core/frame.py:12423, in DataFrame.apply(self, func, ax
  12409         raise ValueError(f"Unknown engine '{engine}'")
  12411     op = frame_apply(
  12412         self,
  12413         func=func,
   (...)  12421         kwargs=kwargs,
  12422     )
> 12423     return op.apply().__finalize__(self, method="apply")
  12424 elif hasattr(engine, "__pandas_udf__"):
  12425     if result_type is not None:

File ~/work/pandas/pandas/pandas/core/apply.py:1015, in FrameApply.apply(self)
   1012 elif self.raw:
   1013     return self.apply_raw(engine=self.engine, engine_kwargs=self.engine_kwargs)
-> 1015 return self.apply_standard()

File ~/work/pandas/pandas/pandas/core/apply.py:1167, in FrameApply.apply_standard(self)
   1165 def apply_standard(self):
   1166     if self.engine == "python":
-> 1167         results, res_index = self.apply_series_generator()
   1168     else:
   1169         results, res_index = self.apply_series_numba()

File ~/work/pandas/pandas/pandas/core/apply.py:1183, in FrameApply.apply_series_generat
   1180 results = {}
   1182 for i, v in enumerate(series_gen):
-> 1183     results[i] = self.func(v, *self.args, **self.kwargs)
```

```
   1184     if isinstance(results[i], ABCSeries):
   1185         # If we have a view on v, we need to make a copy because
   1186         #  series_generator will swap out the underlying data
   1187         results[i] = results[i].copy(deep=False)

Cell In[25], line 2, in f(s)
      1 def f(s):
----> 2     s.pop("a")
      3     return s

File ~/work/pandas/pandas/pandas/core/series.py:5821, in Series.pop(self, item)
   5790 def pop(self, item: Hashable) -> Any:
   5791     """
   5792     Return item and drops from series. Raise KeyError if not found.
   5793
   (...)   5819     dtype: int64
   5820     """
-> 5821     return maybe_unbox_numpy_scalar(super().pop(item=item))

File ~/work/pandas/pandas/pandas/core/generic.py:839, in NDFrame.pop(self, item)
    838 def pop(self, item: Hashable) -> Series | Any:
--> 839     result = self[item]
    840     del self[item]
    842     return result

File ~/work/pandas/pandas/pandas/core/series.py:959, in Series.__getitem__(self, key)
    954     key = unpack_1tuple(key)
    956 elif key_is_scalar:
    957     # Note: GH#50617 in 3.0 we changed int key to always be treated as
    958     #  a label, matching DataFrame behavior.
--> 959     return self._get_value(key)
    961 # Convert generator to list before going through hashable part
    962 # (We will iterate through the generator there to check for slices)
    963 if is_iterator(key):

File ~/work/pandas/pandas/pandas/core/series.py:1046, in Series._get_value(self, label,
   1043     return self._values[label]
   1045 # Similar to Index.get_value, but we do not fall back to positional
-> 1046 loc = self.index.get_loc(label)
   1048 if is_integer(loc):
   1049     return self._values[loc]

File ~/work/pandas/pandas/pandas/core/indexes/base.py:3648, in Index.get_loc(self, key)
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'a'
```

To resolve this issue, one can make a copy so that the mutation does not apply to the container being iterated over.

```
In [28]: values = [0, 1, 2, 3, 4, 5]

In [29]: n_removed = 0

In [30]: for k, value in enumerate(values.copy()):
   ....:     idx = k - n_removed
   ....:     if value % 2 == 1:
   ....:         del values[idx]
   ....:         n_removed += 1
   ....:     else:
   ....:         values[idx] = value + 1
   ....:

In [31]: values
Out[31]: [1, 3, 5]
```

```
In [32]: def f(s):
   ....:     s = s.copy()
   ....:     s.pop("a")
   ....:     return s
   ....:

In [33]: df = pd.DataFrame({"a": [1, 2, 3], 'b': [4, 5, 6]})

In [34]: df.apply(f, axis="columns")
Out[34]:
   b
0  4
1  5
2  6
```

Missing value representation for NumPy types

`np.nan` as the `NA` representation for NumPy types

For lack of `NA` (missing) support from the ground up in NumPy and Python in general, `NA` could have been represented with:

- A *masked array* solution: an array of data and an array of boolean values indicating whether a value is there or is missing.

- Using a special sentinel value, bit pattern, or set of sentinel values to denote `NA` across the dtypes.

The special value `np.nan` (Not-A-Number) was chosen as the `NA` value for NumPy types, and there are API functions like `DataFrame.isna()` and `DataFrame.notna()` which can be used across the dtypes to detect NA values. However, this choice has a downside of coercing missing integer data as float types as shown in Support for integer NA.

`NA` type promotions for NumPy types

When introducing NAs into an existing `Series` or `DataFrame` via `reindex()` or some other means, boolean and integer types will be promoted to a different dtype in order to store the NAs. The promotions are summarized in this table:

Typeclass	Promotion dtype for storing NAs
`floating`	no change
`object`	no change
`integer`	cast to `float64`
`boolean`	cast to `object`

Support for integer `NA`

In the absence of high performance `NA` support being built into NumPy from the ground up, the primary casualty is the ability to represent NAs in integer arrays. For example:

```
In [35]: s = pd.Series([1, 2, 3, 4, 5], index=list("abcde"))

In [36]: s
Out[36]:
a    1
b    2
c    3
d    4
e    5
dtype: int64

In [37]: s.dtype
Out[37]: dtype('int64')

In [38]: s2 = s.reindex(["a", "b", "c", "f", "u"])

In [39]: s2
Out[39]:
a    1.0
b    2.0
c    3.0
f    NaN
u    NaN
dtype: float64

In [40]: s2.dtype
Out[40]: dtype('float64')
```

This trade-off is made largely for memory and performance reasons, and also so that the resulting `Series` continues to be “numeric”.

If you need to represent integers with possibly missing values, use one of the nullable-integer extension dtypes provided by pandas or pyarrow

- `Int8Dtype`
- `Int16Dtype`
- `Int32Dtype`
- `Int64Dtype`
- `ArrowDtype`

```
In [41]: s_int = pd.Series([1, 2, 3, 4, 5], index=list("abcde"), dtype=pd.Int64Dtype())

In [42]: s_int
Out[42]:
a    1
b    2
c    3
d    4
e    5
dtype: Int64

In [43]: s_int.dtype
Out[43]: Int64Dtype()

In [44]: s2_int = s_int.reindex(["a", "b", "c", "f", "u"])

In [45]: s2_int
Out[45]:
a       1
b       2
c       3
f    <NA>
u    <NA>
dtype: Int64

In [46]: s2_int.dtype
Out[46]: Int64Dtype()

In [47]: s_int_pa = pd.Series([1, 2, None], dtype="int64[pyarrow]")

In [48]: s_int_pa
Out[48]:
0       1
1       2
2    <NA>
dtype: int64[pyarrow]
```

See Nullable integer data type and PyArrow Functionality for more.

© 2026, pandas via NumFOCUS, Inc. Hosted by OVHcloud.

Built with the PyData Sphinx Theme 0.16.1.

Created using Sphinx 9.0.4.